

WEEK 09 · DOM, EVENTS, DEBUGGING

THE PAGE STARTS ANSWERING

Last week the language. This week it touches the document — and everything that was theory becomes something you can see going wrong, and find on purpose.

WHERE WE ARE

Week 9 of 12. Three static pages and an array that cannot reach them.

THE NEXT 180 MINUTES

Five cycles, you typing in every one: render · a form that fails well · one listener · Claude's two mistakes · one bug, six steps. Break at 85; the Debug Challenge at 140.

WHAT THIS UNLOCKS

Your list renders from data, adds and removes — and week 10 gives that shape a name.

*“The DOM is a tree-like structure created by the browser that represents the HTML elements **as JavaScript objects.**”*

LESSON 9 OF THE PREVIOUS COURSE, KEPT VERBATIM — THE LAST FOUR WORDS ARE THE ONES DOING THE WORK

TARGET · MINUTE 0

THIS IS WHERE YOUR OWN LIST WILL BE BEFORE LESSON 10

a reading list – one entity of many; yours looks like your project

reload open

סיכום פריט הרשימה מדף הקריאה

הספרים שלי

שם הספר שנת הוצאה

הוסף

מספר ספרים: 3

חומה של דיו

המנהרה

הספרים שלי

שם הספר שנת הוצאה

הוסף

מספר ספרים: 3

חומה של דיו

המנהרה

targets/week-09.png · 1280

סיכום פריט הרשימה מדף הקריאה

הספרים שלי

שם הספר

375 · one column, nothing overflows

► **THE POINT** — a list drawn from an array, a form that adds and fails well, and **ONE** listener that removes any row, including one added a second ago. You type each piece once in a cycle, on another page. At 140 the Debug Challenge starts: **35 MINUTES, ALONE.**

AI THIS WEEK

PARTNER. IT WRITES; YOU OWN EVERY LINE.

LAYER · WEEKS 8–12 · PARTNER

ALLOWED

Code from a rule you wrote down — the quantity check in cycle 2, the remove button in cycle 4 — in Claude Code or in the chat. Then you run it against the rules of the week, in the browser, not against its explanation.

NOT ALLOWED

Any AI in the Debug Challenge. 35 minutes, no AI, no search, no neighbour — it is week 13 in miniature. And keeping code you did not run.

NO SUBSCRIPTION?

Everything works in the free chat, and nothing in the course depends on a paid plan. Paste the file, copy back what returns, do the same review.

CYCLE 1 · 25 MIN

RENDER FROM THE ARRAY

A tool shelf whose three rows were typed into the HTML, and an array with the same three tools. Delete the rows; build each one with `createElement`; let `render()` empty the list, fill it, and derive the empty state and the count. A fourth tool in the array appears on its own.

1 I DO

10 MIN

3 on the next four slides, 7 in the editor · live-code/01-render-from-the-array · delete the li · `toolRow` without its return (wrong) · `render · empty.hidden = tools.length > 0`

2 WE DO

5 MIN

you type steps 1, 3, 4, 5 · four rows, 4 tools, no li in the HTML · compare with checkpoint 1

3 YOU DO

8 MIN

a tool with 0 on the shelf gets is-out with `classList` · then empty the array and read the page

4 DEBRIEF

2 MIN

`append(undefined)` did not complain · ↗ reference

YOU DO · AI-FREE

YOUR THREE PAGES CANNOT CHANGE

WHAT YOU BUILT IN WEEK 8

Three items, written into `index.html` by hand. They look right.

WHAT YOU ALSO HAVE

An array of items in a module, with the same three things in it.

THE PROBLEM

They are two copies of the same truth, and nothing keeps them together. Add a fourth item to the array and the page does not care.

A page that cannot be changed by code is a picture of an application.

BUILDING A ROW, ONE OBJECT AT A TIME

```
1  const li = document.createElement('li');
2  li.dataset.id = item.id;           // <li data-id="a1">
3
4  const title = document.createElement('span');
5  title.className = 'item-title';
6  title.textContent = item.title;    // TEXT. never parsed as markup.
7
8  const remove = document.createElement('button');
9  remove.type = 'button';
10 remove.className = 'remove';
11 remove.append('הסר');
12 li.append(title, remove);          // append takes several at once
```

Nothing here is on the page yet.
It exists, and it is not attached.

render() : EMPTY IT, FILL IT, DERIVE THE REST

```
1 function render() {  
2   shelf.replaceChildren(); // empty the mount  
3   for (const tool of tools) shelf.append(toolRow(tool));  
4  
5   empty.hidden = tools.length > 0; // the empty state is DERIVED  
6   count.textContent = tools.length; // so is the count  
7 }
```

DERIVED, IN ONE PLACE

No handler ever switches `#empty` on or off. It is a fact about the array, so `render` reads it off the array — every time, and therefore never wrong.

REMEMBERED, IN THREE PLACES

A handler that adds shows the list, a handler that removes checks for zero, and one day a third path forgets. Two copies of the truth again.

THE RULE, IN ONE LINE

THE BUG

```
1 const review = form.review.value;
2 box.innerHTML = `<p>${review}</p>`;
3
4 // 
6 // runs with the logged-in user's session.
```

THE FIX

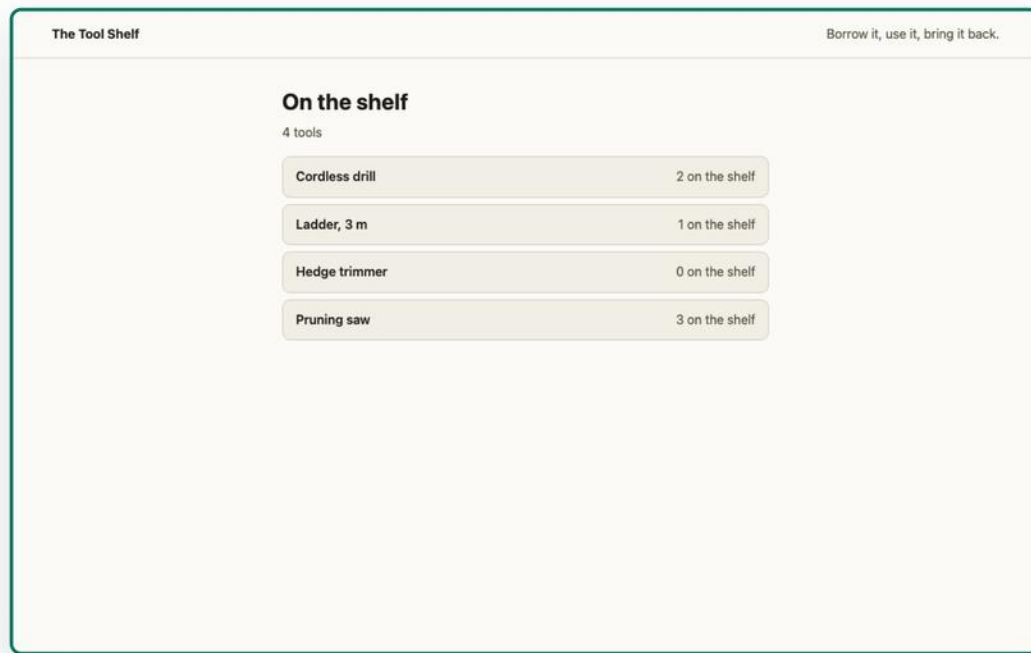
```
1 const p = document.createElement('p');
2 p.textContent = review;
3 box.replaceChildren(p);
4
5 // the structure comes from createElement,
6 // the text comes from the user. never mixed.
```

Text a person typed never reaches the HTML parser. That is the whole rule.



CHECKPOINT 1

YOUR SCREEN SHOULD LOOK LIKE THIS



Compare — `live-code/01-render-from-the-array/checkpoint.png`. Four rows and 4 tools, and in `index.html` not a single `<li>`. In Elements: four `li` with `data-id`, and `#empty` carrying `hidden`.

CYCLE 2 · 25 MIN

THE FORM THAT FAILS WELL

A form that adds without a reload, converts the quantity once at the boundary, resets and returns the cursor — and fails with a sentence, a mark on the field and the focus. Then one keyboard shortcut: a key does exactly what its button does, and never while you type in a field.

1 I DO

10 MIN

3 on the next four slides, 7 in the editor · live-code/02-the-form-that-fails-well · submit + preventDefault · Number() at the boundary · form.reset() · three things on failure · keydown without its guard (wrong) · the guard · a flash that clears itself (setTimeout)

2 WE DO

5 MIN

you type steps 1, 2, 3, 5, 6 · Enter adds, "a" fails well, "Spanner set" types whole · compare with checkpoint 2

3 YOU DO

8 MIN

the quantity: an empty field adds a 0 — refuse it · AI as partner: two lines of spec, then three inputs, not the code

4 DEBRIEF

2 MIN

Number('') is 0, not NaN · ↗ reference

THE DEFAULT ACTION IS A PAGE LOAD

left: no preventDefault, in a frame so the demo survives it

reload

open

THE DEFAULT ACTION IS A PAGE LOAD · THE VALUE IS ALWAYS A STRING

NO PREVENTDEFAULT — IN A FRAME, SO IT CAN SURVIVE

note
type something **Save**

this page has loaded 1 time(s)
location.search (empty)
the field now holds (empty — what you typed is gone)

No preventDefault anywhere in this page.

Type something and press Save. The counter goes up: the browser asked the server for that page again. Every variable that page held is gone, which is why a half-finished list disappears.

HANDLED: PREVENTDEFAULT, VALIDATE, RESPOND

item quantity **Add**

The page never reloads, the list is built from a JavaScript array, and a failed submit says **which field** and **why** — then puts the cursor in it.

EXPRESSION	RESULT	WHY
<code>typeof qty.value</code>	<code>"string"</code>	the DOM stores what was typed, as text
<code>qty.value === 5</code>	<code>false</code>	a string is never strictly equal to a number
<code>qty.value == 5</code>	<code>false</code>	loose equality converts the string first
<code>qty.value > 3</code>	<code>false</code>	a comparison operator converts too — which is why the bug survives testing
<code>Number(qty.value)</code>	<code>1</code>	the conversion you should write yourself
<code>Number.isInteger(Number(qty.value))</code>	<code>true</code>	what a quantity check actually needs

► **TYPE SOMETHING, PRESS SAVE, AND ASK WHAT JUST HAPPENED.** The answer is usually “nothing” or “it flashed”.

`input.value` IS A STRING. ALWAYS.

```
1  const qty = document.querySelector('#qty'); // <input type="number">
2  typeof qty.value // "string"
3
4  qty.value === 5 // false - a string is never === a number
5  qty.value == 5 // true - loose equality converts
6  qty.value > 3 // true ← AND THIS IS WHY THE BUG SURVIVES
7
8  qty.value + 1 // "51" - plus with a string concatenates
9  Number(qty.value) + 1 // 6
10
11 Number('') // 0, not NaN. an empty field passes "is it a number".
```

**Convert once, at the boundary,
where the string comes in.**

FAILING WELL IS THREE THINGS, NOT ONE

```
1 form.addEventListener('submit', (event) => {
2   event.preventDefault();
3   const name = nameField.value.trim();
4
5   if (name.length < 2) {
6     error.textContent = 'A tool name needs at least two characters.'; // WHAT is wrong
7     nameField.setAttribute('aria-invalid', 'true'); // WHICH field
8     nameField.focus(); // WHERE to type
9     return;
10  }
11  nameField.setAttribute('aria-invalid', 'false'); // and CLEAR it when it passes
12  error.textContent = '';
13
14  tools.push({ id: `t${nextId}`, name, qty });
15  form.reset(); // every field back to its initial value
16  nameField.focus();
17  render();
18 });
```

MESSAGES THAT ARE WORTH ZERO

“Error” · “Invalid input” · “Please fill in all fields”. All true. None of them can be acted on.

A KEY DOES WHAT ITS BUTTON DOES — EXCEPT WHILE YOU TYPE

```
1 document.addEventListener('keydown', (event) => {
2   if (event.target.matches('input, textarea, select')) return; // typing is typing
3
4   const button = [...document.querySelectorAll('[data-key]')].find(
5     (candidate) => candidate.dataset.key === event.key,
6   );
7   if (!button) return;
8   event.preventDefault();
9   button.click(); // the same function the mouse runs, not a copy of it
10 });
```

WITHOUT LINE 2

The listener sits on `document`, so it hears typing in the form too. Type `Spanner set` in the name field and the field says **Spaer set**. No error anywhere.

HOME ASSIGNMENT 3

A key plays its pad, **not while you type a name**. And the pad flashes like New tool in step 6:

```
pad.classList.add('hit');
setTimeout(() => pad.classList.remove('hit'), 150);
```

CHECKPOINT 2

YOUR SCREEN SHOULD LOOK LIKE THIS — AND YOUR HANDS SHOULD CHECK MORE

The Tool Shelf

New tool

On the shelf

Tool

How many

Add to the shelf

4 tools

Cordless drill	2 on the shelf
Ladder, 3 m	1 on the shelf
Hedge trimmer	0 on the shelf
Pruning saw	3 on the shelf

Compare, then test — Enter in a field adds a row and the address bar does not change · a fails with a sentence, a red border and the cursor in the field ·
Spanner set types whole · n outside a field jumps to the form.

CYCLE 3 · 25 MIN

DELEGATE THE REMOVE

A Remove button on every row, with a span inside it. The loop that gives each button its own listener works once and then never again; one listener on the list, `closest()` from whatever was hit, and a handler that changes the array and calls `render()` — never draws.

1 I DO

10 MIN

3 on the next four slides, 7 in the editor · live-code/03-delegate-the-remove · the button · one listener per button at load (wrong) · one listener on the list, `closest()`

2 WE DO

5 MIN

you type steps 1 and 3 · add, remove what you added, remove them all · compare with checkpoint 3

3 YOU DO

8 MIN

lab/warmup/ — five functions, four wrong, the referee says what it got and never which line

4 DEBRIEF

2 MIN

target is what was hit, `currentTarget` is who listens · ↗ reference

YOU DO · AI-FREE

A LISTENER IS ATTACHED TO AN ELEMENT, NOT TO A SELECTOR

THE BUG

```
1 for (const b of list.querySelectorAll('.remove')) {  
2   b.addEventListener('click', onRemove);  
3 }  
4  
5 // runs ONCE, over the elements that exist NOW.  
6 // a row added later has no listener.  
7 // and after render() every row is a new object.
```

THE FIX

```
1 list.addEventListener('click', (event) => {  
2   const button = event.target.closest('.remove');  
3   if (!button) return;  
4   onRemove(button.dataset.id);  
5 });  
6  
7 // one listener, attached once, to a container  
8 // that render() empties but never replaces.  
9 // 'change' on a checkbox bubbles too: the same  
10 // delegation works for it.
```

DELEGATION, MEASURED

same array, same render, one listener in a different place

↺ reload

open ↗

SAME ARRAY · SAME RENDER · ONE LISTENER IN A DIFFERENT PLACE

add an item

re-render both

reset

☐ re-bind after every render (the usual "fix")

One listener per button

```
list.querySelectorAll('.remove')
  .forEach(b => b.addEventListener('click', ...))
```

Burr grinder

remove

Gooseneck kettle

remove

Bench scale

remove

addEventListener calls so far 0

rows on screen 3

rows whose button is wired 0 of 3

clicks this list heard 0

One listener on the list

```
list.addEventListener('click', (e) => {
  const b = e.target.closest('.remove'); ... })
```

Burr grinder

remove

Gooseneck kettle

remove

Bench scale

remove

addEventListener calls so far 1

rows on screen 3

rows whose button is wired all 3, by one listener

clicks this list heard 0

Both lists work right now. Press **add an item**, then try to remove the yellow row on each side.

► **"ADD AN ITEM" KILLS ONE ROW. "RE-RENDER" KILLS ALL OF THEM.** Two buttons, two different bugs.

THE OBJECT THE BROWSER HANDS YOU

```
1 list.addEventListener('click', (event) => {  
2   // one argument, and it is not optional reading  
3  
4   event.type           // 'click'  
5   event.target         // WHAT WAS HIT – often a span inside a button  
6   event.currentTarget  // WHO IS LISTENING – always `list` here  
7   event.key            // on a keyboard event: 'Enter', 'Escape', 'a'  
8  
9   event.preventDefault(); // do not do the browser's default thing  
10  event.stopPropagation(); // do not let anyone above hear this  
11 });
```

THE TWO THAT GET CONFUSED, AND IT IS THE BUG IN THIS WEEK'S CHALLENGE

`target` changes with every click. `currentTarget` is the element you attached to, and it never changes.

A HANDLER CHANGES THE DATA AND CALLS `render()` . IT NEVER DRAWS.

```
1 shelf.addEventListener('click', (event) => {  
2   const button = event.target.closest('[data-action="remove"]');  
3   if (!button) return;           // not a Remove button  
4  
5   const id = button.closest('li').dataset.id;  
6   tools = tools.filter((tool) => tool.id !== id); // change the data...  
7   render();                          // ...and draw from it  
8 });
```

THE TELL THAT A HANDLER IS DRAWING

`row.remove()` , `classList` or `textContent` in the body of a listener. It works today; it is a second copy of the truth, and in week 11, when the array is saved, the two copies drift apart.

CHECKPOINT 3

YOUR SCREEN SHOULD LOOK LIKE THIS

The Tool Shelf

New tool

On the shelf

Tool

How many

Add to the shelf

4 tools

Cordless drill

2 on the shelf

Remove

Ladder, 3 m

1 on the shelf

Remove

Hedge trimmer

0 on the shelf

Remove

Pruning saw

3 on the shelf

Remove

Compare, then test — remove two in a row · add Wheelbarrow and remove it · click a tool's name and nothing happens · remove them all: "The shelf is empty" and `0`, without a line that switches it on.

BREAK

10 MINUTES

Back at 95. Cycle 4 is the AI cycle — a chat window open, free or paid,
Live Server running, and

`ai-lab/04-catch-two-mistakes/artifact/start/` in a tab.

CYCLE 4 · 25 MIN

"ADD A REMOVE BUTTON TO EACH ITEM" — AND THE TWO MISTAKES THAT COME BACK

A small watch-later list, built with innerHTML, pasted to Claude with a one-line request. The answer works: a Remove on every row, a click removes. You type a title into its form, open Event Listeners, and find what nobody asked it to check.

1 I DO

8 MIN

3 on the next slide, 5 in the chat ·
ai-lab/04-catch-two-mistakes · P1
live · run what came back · it works
· no names yet

2 WE DO

6 MIN

P1 in your own chat · a title with
onerror, then remove another row ·
Event Listeners on a button · find
both before they get a name

3 YOU DO

9 MIN

your own P2 with constraints you
can check · run it · one journal
entry, after the good example is on
screen

4 DEBRIEF

2 MIN

it answered the question you asked
· ↗ reference

YOU DO · AI AS PARTNER

WHAT CAME BACK

```
1 function render() {
2   queue.innerHTML = talks
3   .map((talk) => `
4     <li>
5       ${talk.title} <span class="mins">${talk.minutes} min</span>
6       <button class="remove-btn" data-id="${talk.id}">Remove</button>
7     </li>`)
8   .join('');
9
10  // Attach a click handler to each remove button
11  document.querySelectorAll('.remove-btn').forEach((btn) => {
12    btn.addEventListener('click', () => {
13      const id = Number(btn.dataset.id);
14      talks = talks.filter((talk) => talk.id !== id);
15      render();
16    });
17  });
18 }
```

► **IT WORKS.** Every row has a Remove; a click removes it; the console is silent; axe finds nothing. Run it before you read it.

TWO MISTAKES, AND NEITHER OF THEM SHOUTS

1 · THE TITLE RUNS

`{{talk.title}}` is what somebody typed into the form, handed to the HTML parser. A title with `onerror` ran when it was added — and **again when another row was removed**, because every render re-parses the whole list. `window.__ran`: 1, then 2.

2 · A LISTENER PER BUTTON, INSIDE RENDER

It works, and it is registered again on every draw: `addEventListener` ran **4, 8, 11** times after load, one add and one removal — for three buttons on screen. Event Listeners shows a `click` on the button and none on the list.

It answered the question you asked.
Nobody asked it to check `render()`.

CYCLE 4 · ON SCREEN BEFORE ANYONE WRITES

WHAT A JOURNAL ENTRY LOOKS LIKE

WHAT WAS WRONG, AND HOW I SAW IT

One row per mistake: **what** I found, **how** I saw it — the title I typed, the number in the console, the tab in Elements — and **why** it is wrong.

WHAT I LEARNED ABOUT THE PROMPT

"Add a button" gets a button. Where the listener lives and what builds a row have to be **in the prompt**, as constraints I can check afterwards.

► **AI-EXEMPLARS/EXEMPLARS/PROMPT-JOURNAL-GOOD.MD** in your course materials. It is graded in your project's `PROMPTS.md` ; today you write one entry, on this cycle.

CHECKPOINT 4 · THREE QUESTIONS

BEFORE YOU LEAVE CYCLE 4

1 Did you type a title with onerror into the P1 answer — and check again after removing another row?

Not "I read the code and it looked fine".

2 In your P2 answer, is there one click listener, on the ul, and no innerHTML in the file?

Elements, Event Listeners — not its explanation.

3 Does your journal line say how you saw each mistake?

The title you typed, the number, the tab.

CYCLE 5 · 20 MIN

ONE BUG, SIX STEPS

The shelf empties and the empty line never appears. Silent, every time. Reproduce, isolate to a line, write a hypothesis that could be wrong, test it with one conditional breakpoint — then fix the symptom by mistake, and let the confirm step catch it.

1 I DO

7 MIN

2 on the next slide, 5 in DevTools and the editor · live-code/05-one-bug-six-steps · a row in notes-he.md per step · the symptom fixed (wrong) · the cause fixed

2 WE DO

5 MIN

the same fault on your copy, six rows, a real conditional breakpoint · compare with checkpoint 5

3 YOU DO

6 MIN

demo 09, failure 3: six rows for a bug whose frame 0 is not where it lives

4 DEBRIEF

2 MIN

confirm is the step that disappears first · ↗ reference

YOU DO · AI-FREE

CYCLE 5 · THE METHOD

FINDING A BUG ON PURPOSE

- 1 **Reproduce**
— what sequence makes it happen, every time?
- 2 **Isolate**
— which file, which function, which line?
- 3 **Hypothesise**
— “I think X is Y when it should be Z.”
- 4 **Verify**
— one breakpoint that makes that sentence true or false.
- 5 **Fix**
— the cause, not the symptom.
- 6 **Confirm**
— redo step 1, and check what is beside it.

► A hypothesis you cannot disprove is not a hypothesis. **IT IS A MOOD.**

`null` IS AN ANSWER, NOT A FAILURE

```
1  const box = document.querySelector('#totl'); // typo
2  box.textContent = '9'; // TypeError: Cannot set properties of null
3
4  // READ THE MESSAGE AS TWO HALVES:
5  //   what was null → box, the result of the selector
6  //   what you did with it → setting "textContent"
7
8  // so the bug is one line ABOVE the line in the trace.
9  // a selector that matched nothing does not complain.
```

The error is where it broke. The bug is where the wrong value was made.



READING A STACK TRACE PROPERLY

four real failures, four real traces↶ reloadopen ↷

FOUR REAL FAILURES · FOUR REAL TRACES

A paragraph with id="target", so failure 1 can be about the selector and not about the page.

1 · a selector that matched nothing

2 · a property of undefined

3 · a guard that threw, three calls deep

4 · inside an event listener

THE ERROR, AS THE CONSOLE SHOWS IT

Press a button.

THE CODE THAT RAN

—

The method, every time.

1. **Reproduce** — what exact sequence makes it happen, every time?
2. **Isolate** — which file, which function, which line?
3. **Hypothesise** — one sentence: "I think X is Y when it should be Z."
4. **Verify** — a breakpoint or one log that makes the hypothesis true or false.
5. **Fix** — change the cause, not the symptom.
6. **Confirm** — redo step 1, and check that nothing near it broke.

► FRAME 0 IS WHERE IT THREW. THE LINE TO OPEN IS THE FIRST FRAME IN A FILE YOU WROTE.

CHECKPOINT 5

YOUR SCREEN SHOULD LOOK LIKE THIS — AND YOUR NOTES SHOULD HOLD SIX ROWS

The Tool Shelf

New tool

On the shelf

Tool

How many

Add to the shelf

4 tools

Cordless drill

2 on the shelf

Remove

Ladder, 3 m

1 on the shelf

Remove

Hedge trimmer

0 on the shelf

Remove

Pruning saw

3 on the shelf

Remove

Compare — the page looks exactly like checkpoint 3, and that is right: the fix changed one word. Your `notes-he.md`: a hypothesis in the form "X is Y when it should be Z", and a test row with the two numbers you saw in Watch. **Before you commit:** remove the breakpoint and every `console.log` you added — what found the bug is not part of the fix.

ASSIGNMENT · 140–175

CLASS ASSIGNMENT 9: 35 MINUTES ALONE, THEN YOUR PROJECT

NOW · 35 MINUTES · ALONE

Part A of the brief, the Debug Challenge: five planted faults in one application. Fix them, and write down **how you found each** in six rows. **No AI, no search, no neighbour.**

AT HOME · BEFORE LESSON 10

Parts B–D of the brief, on your own entity: cycles 1, 2 and 3 assembled — the list from the array, the form that fails well, one listener that removes. **Delegation is required, not suggested.**

HOME ASSIGNMENT 3

Out since week 8, due before lesson 10: pads built from an array, a key that does what its pad does — not while you type — and one listener for the history.

► **THE WORKSHEET IS HALF THE MARKS OF PART A**, and the row that decides is “how did you test it” — exactly the fourth row of cycle 5.

THREE QUESTIONS BEFORE YOU GO

- 1 Two of the five faults shouted and three were silent.
Which silent one cost you the most time — and which of the six steps finally found it?
- 2 A title typed into a form reaches `innerHTML`.
When does it run — and when does it run again?
- 3 Ten rows, ten listeners attached at load. A row is added, then one is removed.
How many rows answer a click now, and why?

GO DEEPER

The reference deck for this week — every slide the lecture pointed at, in order.

[3 · The file and the document are two different things](#) ← spine 4

[5 · The script that ran before the page existed](#) ← spine 4

[6 · You already know the selector language](#) ← spine 4

[7 · Selecting](#) ← spine 4

[11 · Traversing: up, and asking questions](#) ← spine 16

[12 · Creating and inserting](#) ← spine 4

[13 · append takes strings too — which is how undefined gets in](#) ← spine 4

[14 · Three ways to add a row. One of them kills your listeners.](#) ← spine 16

[17 · Emptying, removing, copying](#) ← spine 4

[20 · The reason, and it is not the speed](#) ← spine 23

[21 · Content: four ways in, and only two you should use](#) ← spine 23

[22 · State goes in classes. Values go in style.](#) ← spine 4

[24 · Classes, styles, attributes, data](#) ← spine 4

[25 · hidden loses to your own display](#) ← spine 4

[26 · The attribute is the file. The property is the element.](#) ← spine 10

[27 · Listening, and the parentheses that ruin it](#) ← spine 10

[28 · The events you will actually use](#) ← spine 10

[29 · Keyboard: key, code, and where typing happens](#) ← spine 10

[31 · The event object](#) ← spine 10

[32 · One click, seven listeners, one order](#) ← spine 16

[35 · Delegation, as a recipe](#) ← spine 16

[36 · Counting the listeners](#) ← spine 23

[37 · Forms](#) ← spine 10

[39 · Elements: the DOM, editable, in front of you](#) ← spine 28

[40 · The console has five more things in it](#) ← spine 28

[41 · Sources: stopping time](#) ← spine 28

[43 · DevTools, by panel](#) ← spine 28

[44 · The six steps, and what each one costs](#) ← spine 28

[45 · Where to look, in order](#) ← spine 28